

Report

Motivazione e scelta dell'ADT:

Per implementare la soluzione è stato utilizzato l'ADT **idalbum**, contenuto nel file "ADT_Album.h" e strutturato come segue:

1. Char *titolo;
2. Char *artista;
3. Char *genere;
4. Int anno.

L'ADT contiene tutti i parametri essenziali e richiesti dalla traccia e viene utilizzato come elemento e supporto per le operazioni che riguardano l'ADT principale.

Nel file "Album.h" è infatti implementata la **lista** che non è nient'altro che un puntatore alla struttura album: **struct album**.

Le strutture album rappresentano i nodi della lista, sono implementati nel file "Album.c" e definiti come segue:

1. Char *titolo;
2. Char *artista;
3. Char *genere;
4. Int anno;
5. Struct album *next: puntatore al prossimo nodo.

Viene utilizzata la lista poiché perfetta per la gestione e la manutenzione di dati dinamici la cui dimensione non è conosciuta a priori e può variare nel tempo.

Le liste sono perfette anche per gestire grandi quantità di dati presi in input sia da file che da linea di comando, possono gestire benissimo anche elementi non ordinati e nel caso ordinarli in seguito.

Questo ADT viene spesso utilizzato in algoritmi di ricerca o ordinamento e proprio per questo perfetto per la ricerca di specifici elementi come album in un catalogo.

Presentano una flessibilità maggiore rispetto ad altre strutture dati ed essendo usato molto per gestioni come rubriche o carrello di acquisti online è predisposta per l'aggiunta e la rimozione di elementi anche in mezzo alla lista, avendo ovviamente le opportune accuratezze.

Progettazione:

L'implementazione garantisce tutti i tipi di **astrazione** offrendo solo l'interfaccia per utilizzare le funzioni senza avere i dettagli implementativi delle varie funzioni e della struttura dati.

Tutto parte dall'header file "ADT_Album.h" e dal file "ADT_Album.c" che predispongono la struttura iniziale **idalbum** utile come appoggio per gli elementi della lista e definiscono anche 2 funzioni utili alle relazioni di queste strutture tra loro, queste funzioni sono:

- void output_id (idalbum *album): funzione al quale viene passato il puntatore di un album in input e stampa tutti i suoi campi;
- int comparaAlbum (idalbum *x, idalbum *y): funzione al quale vengono passati i puntatori di 2 strutture idalbum e compara i campi 1 ad 1 e in confronto all'esito della comparazione restituisce una variabile booleana 1 o 0.

Questo modulo viene incluso poi in un altro modulo "Album.h" che definisce la **lista** come puntatore a struct album, proprio per assicurare che l'utente non abbia diretto accesso ai campi della struttura per garantire l'**information hiding** e definisce delle macro utili al codice sorgente come

Report

ANNO_CORRENTE impostato a 2024 e MAX_LEN che rappresenta la lunghezza massima che può avere una stringa, ovvero 50.

Nel modulo sono precisati i prototipi delle varie funzioni di cui l'utente può usufruire:

- lista [nuovaLista](#) (void): che restituisce una lista vuota;
- int [listaVuota](#) (lista l): restituisce un valore di verità booleano che indica se la lista è vuota o meno;
- lista [aggiungiAlbum](#) (lista l, char *tit, char *art, char *gen, int anno): funzione che prende in input la lista e i vari parametri da assegnare ai campi della struttura e aggiunge un nuovo nodo alla lista restituendo il nuovo puntatore;
- lista [rimuoviAlbum](#) (lista l, int pos): prende in input la lista e la posizione da eliminare e restituisce il nuovo puntatore alla lista;
- int [modificaAlbum](#) (lista l): prende in input la lista, modifica il campo desiderato e restituisce un intero che indica l'esito dell'operazione;
- lista [codaLista](#) (lista l): restituisce il puntatore al prossimo album nella lista;
- idalbum* [prendiPrimo](#) (lista l): restituisce il puntatore al primo album del catalogo, ovvero l'elemento in testa alla lista;
- int [grandezzaLista](#) (lista l): restituisce il numero di album contenuti nel catalogo;
- lista [invertiLista](#) (lista l): restituisce il puntatore di una lista che è il contrario di quella passata in input, ovvero il primo elemento della nuova lista è l'ultimo di quella passata e così via;
- int [posizioneAlbum](#) (lista l, idalbum *x): avendo implementato la rimozione tramite posizione nella lista serviva una funzione che restituisse la posizione dell'album cercato, ed è proprio quello che fa questa lista, prendendo in input la lista e l'album cercato;
- lista [cercaAlbum](#) (lista l): funzione che prende in input la lista e restituisce una nuova lista con tutti gli album contenenti il valore specifico del campo cercato;
- lista [fcercaAlbum](#) (FILE *f, lista l): stesso utilizzo della funzione precedente con la differenza che l'input viene preso dal file passato come parametro;
- int [cercaTitolo](#) (lista l, char *tit): essendo i titoli degli album usati come "identificatori" dell'album, questa funzione controlla che un determinato titolo (stringa passata come parametro) non sia già presente nel catalogo restituendo un valore booleano che indica l'esito del controllo;
- void [stampaCatalogo](#) (lista l): funzione che serve per stampare tutti gli album del catalogo, non restituisce nulla;
- void [fstampaCatalogo](#) (FILE *f, lista l): funzione che stampa tutti gli album del catalogo in un file passato come parametro di input;
- //void [scambia](#) (lista l1, lista l2): funzione definita nel codice sorgente ma non definita nel modulo poiché non disponibile all'utente, ma usata per facilitare delle operazioni altresì ripetitive, proprio per questo viene presentata ma come commento;
- lista [ordinannoCatalogo](#) (lista l): funzione che presa in input la lista restituisce il puntatore della stessa lista ma ordinata secondo il campo dell'anno in ordine decrescente. Questa funzione sfrutta l'ultima [sopra citata](#).

Nel file "Album.c" è incluso l'header file corrispettivo e definisce la struttura struct album come [sopra indicato](#).

Nel file .c sono presenti tutte le implementazioni adoperate per le funzioni con tanto di commento e spiegazione dei vari passaggi effettuati.

Report

Nel codice sorgente "main.c" è incluso il modulo "Album.h" con tutte le varie librerie utili, ed è definita una costante MAX che indica il numero di caratteri massimo che possono essere immagazzinati in una stringa che rappresenta le righe dei file.

È diviso principalmente in due parti e all'inizio del programma viene chiesto all'utente quale delle due interfacce utilizzare:

1. Prima interfaccia predisposta per la lettura dei dati, la scelta dell'operazione e tutto ciò che ne consegue esclusivamente da file.
Questo tipo di interfaccia viene utilizzata principalmente per il testing del programma per verificare che il suo comportamento sia conforme a quello atteso.
La linea di comando viene utilizzata solo per piccoli accorgimenti quali, input dei nomi dei file o per correggere input di dati che da file non sono conformi alle regole fissate.
Questa interfaccia permette esclusivamente queste operazioni sul catalogo:
 - a. Aggiunta di album da file;
 - b. Rimozione di album da file;
 - c. Ricerca di album da file;
 - d. Stampa del catalogo nel file di output.
2. Seconda interfaccia utilizzata per rapportarsi direttamente con l'utente e lasciargli il libero arbitrio su tutte le operazioni disponibili che vuole effettuare nel catalogo.
Queste operazioni sono ben definite e guidano l'utente sull'uso del programma, una volta terminata l'esecuzione di un'operazione l'utente può decidere se continuare ed effettuare altre operazioni o chiudere il programma.
Questa interfaccia è più ampia, permettendo queste operazioni:
 - a. Aggiunta di album al catalogo;
 - b. Rimozione di album dal catalogo;
 - c. Modifica di campi specifici di un album;
 - d. Trova la posizione di un album in lista;
 - e. Cerca un album;
 - f. Riordina il catalogo per anno in modo decrescente;
 - g. Stampa del catalogo;
 - h. Chiusura del programma.

La funzione main al suo termine restituisce 0 se tutte le operazioni sono state eseguite correttamente e hanno avuto successo, mentre restituisce 1 in caso di errori, principalmente di allocazione o di apertura file.

Specifica Sintattica e Semantica:

Modulo "ADT_Album.h".

Specifica Sintattica

- Tipo di riferimento: idalbum;
- Tipi usati: void, boolean;
- Operatori:
 - output_id (idalbum*) -> void
 - comparaAlbum (idalbum*, idalbum*) -> boolean

Specifica Semantica

- idalbum è la struttura che rappresenta i campi titolo, artista, genere di tipo char* e anno di tipo int;
- Operatori:
 - output_id (album) -> void
 - Pre-condizione: album è un puntatore ad una struttura idalbum correttamente allocata;
 - Post-condizione: non restituisce nulla;

Report

- comparaAlbum (x, y) -> 0/1
 - Pre-condizione: x e y sono puntatori a strutture idalbum correttamente allocati;
 - Post-condizione: se x==y allora 0 altrimenti 1;

Modulo “**Album.h**”.

Specifica Sintattica

- Tipo di riferimento: lista;
- Tipi usati: void, boolean, int, char*, idalbum*, FILE*;
- Operatori:
 - nuovaLista () -> lista
 - listaVuota (lista) -> boolean
 - aggiungiAlbum (lista, char *, char *, char *, int) -> lista
 - rimuoviAlbum (lista, int) -> lista
 - modificaAlbum (lista) -> int
 - codaLista (lista) -> lista
 - prendiPrimo (lista) -> idalbum*
 - grandezzaLista (lista) -> int
 - invertiLista (lista) -> lista
 - posizioneAlbum (lista, idalbum *) -> int
 - cercaAlbum (lista) -> lista
 - fcercaAlbum (FILE *, lista) -> lista
 - cercaTitolo (lista, char *) -> boolean
 - stampaCatalogo (lista) -> void
 - fstampaCatalogo (FILE *, lista) -> void
 - //scambia (lista, lista) -> void
 - ordinannoCatalogo (lista) -> lista

Specifica Semantica

- Lista è l'insieme delle sequenze $L = a_1, a_2, \dots, a_n$ di tipo idalbum;
- L'insieme contiene NULL, che rappresenta la lista vuota;
- Operatori:
 - nuovaLista () -> l
 - Pre-condizione:
 - Post-condizione: $l = \text{NULL}$
 - listaVuota (l) -> 1/0
 - Pre-condizione:
 - Post-condizione: se $l = \text{NULL}$ allora 1 altrimenti 0
 - aggiungiAlbum (l, tit, art, gen, anno) -> l
 - Pre-condizione: $\text{tit} \neq \text{NULL}$, $\text{art} \neq \text{NULL}$, $\text{gen} \neq \text{NULL}$, $1800 \leq \text{anno} \leq \text{ANNO_CORRENTE}$ e $l = \langle a_1, a_2, \dots, a_n \rangle$
 - Post-condizione: $l = \langle \text{nuovo}, a_1, a_2, \dots, a_n \rangle$
 - rimuoviAlbum (l, pos) -> lout
 - Pre-condizione: $l = \langle a_1, a_2, \dots, a_{\text{pos}-1}, a_{\text{pos}}, a_{\text{pos}+1}, \dots, a_n \rangle$ $n > 0$ e $\text{pos} > 0$
 - Post-condizione: $l = \langle a_1, a_2, \dots, a_{\text{pos}-1}, a_{\text{pos}+1}, \dots, a_n \rangle$
 - modificaAlbum (l) -> 0/1/2
 - Pre-condizione: $l = \langle a_1, \dots, a_i, \dots, a_n \rangle$ $n > 0$
 - Post-condizione: un campo di un elemento a_i con $i > 0$ è stato modificato
 - codaLista (l) -> temp

Report

- Pre-condizione: $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
- Post-condizione: $I = \langle a_2, \dots, a_n \rangle$
- [prendiPrimo](#) (I) -> alb
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione: $alb = a_1$
- [grandezzaLista](#) (I) -> n
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_m \rangle, m > 0$
 - Post-condizione: $n = m$
- [invertiLista](#) (I) -> inv
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione: $inv = \langle a_n, \dots, a_2, a_1 \rangle$
- [posizioneAlbum](#) (I, x) -> pos
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_i, \dots, a_n \rangle, n > 0$ e $x = a_i$
 - Post-condizione: $pos = i$
- [cercaAlbum](#) (I) -> I trovati
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione: $I trovati = \langle a_i, a_{i+1}, \dots, a_k \rangle, k > i$ con $a_i, \dots, a_k$ aventi uno specifico valore di un campo
- [fcercaAlbum](#) (f, I) -> I trovati
 - Pre-condizione: f puntatore a file correttamente aperto e $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione: $I trovati = \langle a_i, a_{i+1}, \dots, a_k \rangle, k > i$ con $a_i, \dots, a_k$ aventi uno specifico valore di un campo
- [cercaTitolo](#) (I, tit) -> f
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_i, \dots, a_n \rangle, n > 0$ e $tit \neq \text{NULL}$
 - Post-condizione: Se $a_i.titolo == tit$ allora $f = 1$ altrimenti $f = 0$
- [stampaCatalogo](#) (I) -> void
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione:
- [fstampaCatalogo](#) (f, I) -> void
 - Pre-condizione: f puntatore a file correttamente aperto e $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione:
 - Side-effect: modifica il file f
- [//scambia](#) (I1, I2) -> void
 - Pre-condizione: $I1 = \langle a_1, a_2, \dots, a_n \rangle, n > 0$ e $I2 = \langle b_1, b_2, \dots, b_m \rangle, m > 0$
 - Post-condizione: $I1 = \langle b_1, a_2, \dots, a_n \rangle, n > 0$ e $I2 = \langle a_1, b_2, \dots, b_m \rangle, m > 0$
- [ordinannoCatalogo](#) (I) -> I
 - Pre-condizione: $I = \langle a_1, a_2, \dots, a_n \rangle, n > 0$
 - Post-condizione: $I = \langle a_i, a_{i+1}, \dots, a_n \rangle$ con $a_i.anno > a_{i+1}.anno$

Razionale dei casi di test:

Sono stati effettuati dei testing per le operazioni [sopra citate](#) per controllare malfunzionamenti o input inattesi.

Il file test_suite.txt contiene l'identificatore del caso di test e un numero che può essere 1, 2 o 3, rispettivamente per aggiunta, rimozione e ricerca.

Questo file viene passato da linea di comando quando si fa partire il programma sul terminale con **./main.exe test_suite.txt** e si invia 1 al programma per dirgli di far partire la [prima interfaccia](#).

Il programma prenderà in input dal file l'identificatore del test case e l'operazione da eseguire.

Report

Nel primo caso il programma prende la stringa "TC1" e ricrea i nomi dei file da cui prendere l'input (TC1_input.txt) e l'oracolo, ovvero l'esito che ci si aspetta dall'esecuzione del programma (TC1_oracolo.txt) e crea, se non già esistente, il file dove scrivere in output (TC1_output.txt).

Dopo aver preso l'input da file, procede con l'elaborazione dell'operazione corrispondente e scrive la lista nel file di output.

Salva in una seconda lista l'input del file oracolo e confronta le due liste con un ciclo di `comparaAlbum()`, definita in "ADT_Album.h", soltanto dopo aver aperto in append, ovvero che scrive sul file partendo dalle ultime cose presenti già nel file di testo, il file (risultati.txt).

Per concludere il programma scrive nel file dei risultati l'identificativo del caso di test e il corrispettivo esito che è **PASS** se la lista di output e quella di oracolo sono uguali, altrimenti **FAIL**.

Il procedimento si ripete finché i casi di test non sono conclusi: nei file inviati è presente un test case per ogni operazione.

Sono stati effettuati diversi test anche per la [seconda interfaccia](#) e ogni controllo e verifica effettuati sono predisposti per offrire un'esperienza migliore all'utilizzatore, dando esito positivo ad ogni operazione effettuata per tutti i valori dati in input.

N.B.: Quando viene chiamata l'operazione di ricerca da file (n. 3) c'è bisogno di dare in input al programma un ulteriore file (nel mio caso cerca.txt) nel quale sono presenti: un intero 1, 2, 3 o 4, rispettivamente per titolo, artista, genere, anno e una stringa che rappresenta la stringa da cercare nella lista, qual ora si scegliesse il n. 4 ovvero l'anno, il secondo parametro viene letto ugualmente come una stringa, ma viene trasformato in intero usando la funzione `atoi()`.

Per qualsiasi input non valido presente nei file il programma chiede all'utente di ri-inserire manualmente l'input errato da linea di comando e solo allora concluderà le operazioni.